Comparing Computer Vision Architectures for Pedestrian Counting at the Toomer's Corner Livestream

Logan Bolton
Auburn University
ldb0046@auburn.edu

Jase Schwanke
Auburn University
jms0285@auburn.edu

Sathvik Prahadeeswaran
Auburn University
srp0061@auburn.edu

Trey Wendell
Auburn University
tdw0042@auburn.edu

Abstract

We study pedestrian counting from a single fixed outdoor camera — the Toomer's Corner public livestream at Auburn University. Because no labelled dataset exists for this scene, we build one: frames are sampled from the livestream, pre-annotated with the open-vocabulary SAM 3 detector using the text prompt "person," and then corrected by human reviewers in Label Studio. We treat counting as an object detection problem: a detector outputs person boxes, and the count is the number of boxes retained after a confidence threshold. We compare three models on the same validation split: a COCO-pretrained YOLO26n (generic zero-shot baseline), SAM 3 itself used directly as a zero-shot open-vocabulary counter, and a YOLO26m fine-tuned on our curated data. A large random hyperparameter search (195 successful trials across eight sweep rounds) selects the final model by validation count MSE. The fine-tuned YOLO26m reaches a per-image count MAE of 1.27 and MSE of 2.79, compared to MAE 3.10 / MSE 15.01 for pretrained YOLO26n and MAE 5.00 / MSE 33.09 for SAM 3 zero-shot — roughly a 5 $\times$ reduction in squared error over the next-best model. The main takeaway is that a small amount of in-domain labelled data, bootstrapped with an open-vocabulary model for labelling, is enough to beat both a generic pretrained detector and the open-vocabulary foundation detector itself on this scene.

1 Introduction

Toomer's Corner is a public intersection at Auburn University that is continuously streamed online. Traffic at the corner changes dramatically by time of day and day of week — from a handful of pedestrians during class hours to dense crowds after sporting events — and an automatic crowd-counting system would let the university quantify foot traffic without storing raw video. The livestream is, however, shot from a single fixed rooftop angle with a wide field-of-view, which is quite different from the clean frontal pedestrian imagery that generic detectors are typically pretrained on. This creates a mismatch: off-the-shelf detectors run on these frames produce many missed detections, especially on distant or partially-occluded pedestrians.

We therefore treat this as a concrete, scoped machine learning problem: given a frame from the Toomer's Corner livestream, predict the number of people visible in it. We compare a generic pretrained detector to one fine-tuned on in-domain frames, measure how each performs along

Preprint. Under review.

accuracy and compute-efficiency axes, and use a large random hyperparameter search to ensure the fine-tuned model is actually well-tuned rather than accidentally configured.

1.1 Problem Definition

- **Input:** a single RGB frame sampled from the Toomer's Corner livestream (typically 1280×720).
- **Output:** a non-negative integer $\hat{y}$ estimating the number of people visible in the frame.
- **Why machine learning:** no closed-form feature can count pedestrians under real-world appearance variation (clothing, pose, lighting, occlusion, scale), so a learned detector is required.
- **Why challenging:** the fixed camera is far from many subjects, producing small-scale targets; crowds frequently self-occlude; lighting changes across the day (direct sun, shadow, night); and the scene contains easily-confused non-person objects such as mannequins in storefronts and reflections in shop windows.

1.2 Project Scope and Objectives

We intentionally constrained the project to a tractable scope: one scene, one task (counting), one detector family (YOLO), with a well-defined baseline vs. fine-tuned comparison. Specifically:

- **Objective 1 — Data:** build a $\geq$400-image labelled dataset of Toomer's Corner frames with person bounding boxes, using SAM 3 to pre-annotate and Label Studio for human review.
- **Objective 2 — Baseline:** evaluate an off-the-shelf pretrained YOLO26n on the same validation split as a zero-effort lower bound.
- **Objective 3 — Fine-tuning:** fine-tune YOLO26s and YOLO26m on the curated data, and tune hyperparameters with a large random search whose primary metric is per-image count MSE (not standard mAP, because the downstream task is counting).
- **Objective 4 — Analysis:** compare the two models on accuracy (count MSE, MAE), detection quality (mAP_{50}), and compute (training time, inference latency), and surface qualitative failure modes.

2 Dataset

2.1 Data Source

Frames were captured from the public Toomer's Corner livestream by `extract_frames.py`, which runs `ffmpeg` against the HLS URL at a fixed sampling rate (1 frame per second) and records the wall-clock capture time in each filename. Sampling sessions covered multiple days and times of day to diversify lighting and crowd density. The resulting raw image directories (`frames_output/stream*/`) each contain several thousand unlabelled frames; we subsampled roughly 500 frames for annotation, discarding obvious duplicates (consecutive frames with no visible motion) and frames with corrupted HLS segments.

2.2 Annotation

Manually drawing boxes for $\sim$15 people per image across 500 images is expensive, so we adopted a semi-automatic pipeline:

1. **SAM 3 pre-annotation.** `sam3/annotate.py` runs SAM 3 (Segment Anything 3, an open-vocabulary detector) with the text prompt `"person"` and a confidence threshold of 0.25. It writes YOLO-format label files (normalized `x_center y_center w h`) to `data/labels/raw/`.
2. **Human review.** `sam3/review.py` launches a local Label Studio server, ingests every image with the SAM 3 boxes as pre-loaded predictions, and four team members jointly correct them: adding missed detections, removing false positives (commonly mannequins, reflections, and tree trunks), and tightening boxes on distant pedestrians. Corrected labels are exported back in YOLO format.
3. **Class schema.** A single class, `person`, is used. Cyclists and riders are labelled as persons. Seated persons are labelled. Strollers, dogs, and vehicles are not.

The pre-annotation step turned out to save a large fraction of annotation time on sparse frames, but on dense frames reviewers still had to add many missed detections by hand — quantitatively, SAM 3's raw labels had a count MAE of 5.0 against the corrected ground truth on the validation split, i.e. SAM 3 alone is not accurate enough for the downstream task.

2.3 Dataset Statistics

The final curated dataset (`data/manual_label_4_16_26_v2`) is split 80/20 into train/validation at the image level by `prepare_dataset.py` with `seed=42`. Splits are image-level (no frame from the same 1 s neighbourhood is allowed to straddle train and val, since consecutive frames are near-duplicates). Basic statistics are summarised in Table 1.

Table 1: Dataset summary. "Avg. count" is the mean number of person boxes per image. "Max count" is the single most crowded frame.

Split	Images	Avg. count	Max count	Notes
Train	404	15.8	38	Used for fine-tuning only
Validation	101	16.4	38	Used for all reported metrics

Per-image counts span 5 to 38, with most frames in the 10–20 range; true total across the validation split is 1659 people. The data contains both sparse frames (individual pedestrians, clear sidewalk) and dense frames (game-day crowds, multiple clusters).

2.4 Data Quality and Preprocessing

Resolution. Frames are stored at native livestream resolution and passed to YOLO at `imgsz=1280` during both training and inference. We briefly compared `imgsz=640` and `960`; both degraded recall on distant/small pedestrians, so `imgsz=1280` was fixed for the remainder of the sweep (see Section 4.2).

Filtering bad frames. Frames with HLS corruption (gray blocks, tear artefacts) were removed during annotation by the reviewer.

Leakage prevention. Because consecutive 1 FPS frames are highly correlated, we avoided placing frames sampled within the same short window on opposite sides of the train/val split. The split is applied after filtering duplicates.

Normalization and augmentation. Pixel normalization is delegated to Ultralytics' YOLO pipeline (scale to $[0, 1]$, standard BGR→RGB). Training-time augmentation was treated as a hyperparameter: mosaic, mixup, colour jitter (HSV), random scale, and random erasing were all included in the search space, with horizontal flip disabled (`fliplr=0`) because the scene has asymmetric storefronts that a flipped model tends to hallucinate boxes near. Close-mosaic was toggled on for the final epochs to stabilize the final weights.

3 Methods

3.1 Task Formulation

We formulate counting as *detection-then-count*: the model produces a list of bounding boxes with confidence scores for the `person` class, a confidence threshold τ is applied, and the count is simply the number of retained boxes. At evaluation time we sweep $\tau \in \{0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.50, 0.60\}$ and report the threshold that minimises count MSE on the validation split, i.e. the threshold is part of the evaluation, not the model. We chose detection-then-count over direct regression because it yields interpretable box-level outputs that the reviewers could inspect to debug failure modes, and because the SAM 3 labelling pipeline produces boxes natively.

3.2 Baseline Model: Pretrained YOLO26n

The baseline is the smallest (nano) YOLO26 checkpoint, `yolo26n.pt`, loaded with no fine-tuning and evaluated on the validation split with `classes=[0]` (COCO person class). It represents what

one would get with zero labelling effort and is the natural "is fine-tuning worth the cost" reference point. Nothing is trained; only the inference-time confidence threshold is tuned on the validation set.

3.3 Fine-tuned Model: YOLO26m

The main model is `yolo26m.pt` (the medium YOLO26 checkpoint) fine-tuned on our curated training split via `ultralytics.YOLO.train()`. All layers are unfrozen; we start from COCO-pretrained weights. Person class only (class index 0). We also trained YOLO26s variants as a middle datapoint; YOLO26n fine-tuning was dropped early in the sweep because every `yolo26n` trial in sweep v5 produced catastrophic count errors, suggesting the nano backbone is too small to resolve distant pedestrians at this scene's scale.

3.4 Third Model Compared: SAM 3 (zero-shot)

Because SAM 3 also produces the pre-annotation labels, it is natural to ask how well SAM 3 performs *directly* as a pedestrian counter — no fine-tuning, no YOLO, just run SAM 3 with the text prompt `"person"` on every validation frame and count the returned boxes. This gives us a three-way comparison: a generic pretrained detector (YOLO26n), an open-vocabulary foundation detector (SAM 3), and our fine-tuned specialist (YOLO26m). SAM 3 is evaluated at its authored confidence threshold of 0.25 (the same threshold used for pre-labelling in `sam3/annotate.py`); since it is not trained on our data, no other knobs are tuned.

3.5 Training Setup

Fine-tuning is driven by `train.py` (for single runs) and `sweep.py` (for hyperparameter search). Per-trial defaults during the search: 50 epochs, early stopping patience 15, image size 1280, batch 16, Ultralytics dataloader with `cache=ram`. Hardware: one NVIDIA GPU per worker process, run with two workers in parallel (`CUDA_VISIBLE_DEVICES=0,1`) so that two trials train concurrently without contending over the same device. Model weights, optimizer type, learning rate schedule, loss weights, and all augmentation probabilities were *not* hand-picked — they are hyperparameters (Section 4.2).

Best trial hyperparameters (sweep v10, trial `1776384144137_w0_0015`): `model=yolo26m.pt`, `imgsz=1280`, `batch=16`, `optimizer=auto`, `lr0=0.01`, `lrf=0.1`, `momentum=0.95`, `weight_decay=1e-3`, `warmup_epochs=1`, `cos_lr=True`, `box=10.0`, `cls=1.0`, `dfl=1.5`, HSV `(0.0, 0.5, 0.4)`, `scale=0.25`, `fliplr=0.0`, `flipud=0.1`, `mosaic=0.5`, `mixup=0.25`, `close_mosaic=5`, `dropout=0.2`.

3.6 Evaluation Metrics

Because counting is the end task, our primary metrics are per-image count errors, not box-level localisation:

- **Count MSE** (primary sweep metric): $\frac{1}{N}\sum_i(\hat{y}_i - y_i)^2$, where $\hat{y}_i$ is the number of surviving boxes and y_i the ground-truth count.
- **Count MAE:** $\frac{1}{N}\sum_i|\hat{y}_i - y_i|$, reported because absolute-error units are easy to reason about ("off by k people per frame").
- **Count bias:** mean signed error, which tells us whether a model systematically under- or over-counts.
- **$\mathbf{mAP_{50}}$:** standard COCO-style mean average precision at IoU = 0.5, reported for context (box-quality, not counting-quality).
- **Training time** per trial (seconds), recorded automatically by the sweep.
- **Inference latency** (seconds) for the full validation split on a single GPU, end-to-end including image decode.

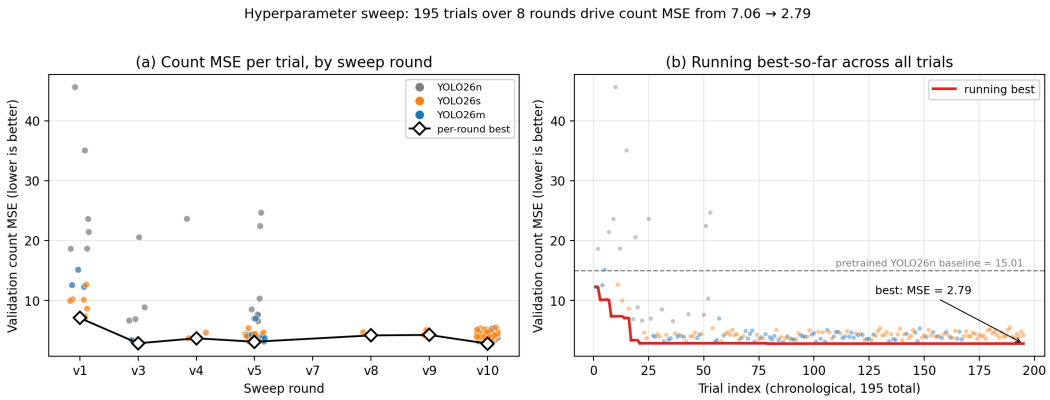

Figure 1: Hyperparameter sweep progression. (a) Each dot is one successful trial; colour encodes YOLO backbone (`yolo26n/s/m`); the black diamond line tracks the best count MSE achieved in each round. Early rounds (`v1`–`v4`) contain many catastrophic `yolo26n` trials; later rounds concentrate on `yolo26m`. (b) Trials plotted in chronological order, with the running best-so-far curve in red and the pretrained YOLO26n baseline (MSE 15.01) as a dashed reference.

4 Experiments

4.1 Main Comparison

All three models — pretrained YOLO26n, zero-shot SAM 3, and fine-tuned YOLO26m — are evaluated on the same validation split using the same preprocessing. For both YOLO models, the confidence threshold is tuned on the validation set by grid-search to minimise count MSE. SAM 3 uses its authored threshold of 0.25. Per-image count errors at each model's selected threshold are reported in Section 5.1.

4.2 Hyperparameter Sweep

`sweep.py` implements a random search over a 26-dimensional space (model size, optimizer, LR schedule, loss weights, augmentation probabilities, regularization). Each trial samples one config, trains one YOLO model to completion with that config, evaluates the best-confidence-threshold count MSE on the validation split, and appends a row to `sweeps/<name>/results.csv`. We ran eight sweep rounds (`v1, v3, v4, v5, v7, v8, v9, v10`) for a total of 236 trials, 195 of which completed successfully — the remainder died on out-of-memory or transient CUDA errors. Between rounds, we pruned the search space using the analysis of per-axis mean count MSE from the previous round (e.g. after v5, we dropped `yolo26n` because every nano trial regressed badly, and pinned several augmentation knobs whose variance across the sweep was dominated by noise). The final sweep `v10` is therefore a tighter, focused search over the parameters that still showed meaningful spread. Figure 1 visualises the progression: per-round spread narrows as the search space tightens, and the running best-so-far curve across all 195 trials decreases from an early-round best of 7.06 to a final best of 2.79 — well below the pretrained-baseline reference line at 15.01.

4.3 Trend Analysis

Frame filenames record the wall-clock capture time (see `extract_frames.py`), so applying the final fine-tuned model to unlabelled frames from a multi-hour stream produces a per-second count time series. We report one such time series in Section 5.3.

Table 2: Main validation-set results. Arrows indicate direction of improvement. Training and inference times are recorded on the same single-GPU setup. Neither baseline (YOLO26n, SAM 3) is trained on our data, so training time is zero.

Model	MAE ↓	MSE ↓	mAP_{50} ↑	Train time	Inference (full val)
Pretrained YOLO26n (baseline)	3.10	15.01	—	0 s	1.3 s / 91 imgs
SAM 3 zero-shot (`"person"`)	5.00	33.09	—	0 s	45 s / 91 imgs
Fine-tuned YOLO26m (ours)	**1.27**	**2.79**	**0.908**	588 s	2.9 s / 91 imgs

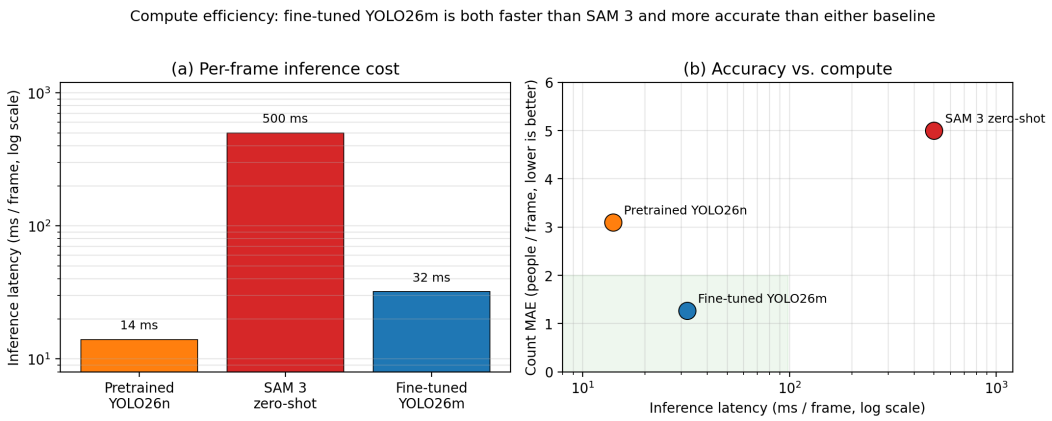

Figure 2: Compute efficiency on the Toomer's Corner validation split. (a) Per-frame inference latency on a single GPU (log scale). (b) Accuracy vs. compute Pareto plot: count MAE (y-axis) against per-frame inference latency (x-axis, log scale). The fine-tuned YOLO26m is the only model in the low-latency, low-error region.

5 Results

5.1 Quantitative Results

Table 2 reports the three-way comparison on the validation split at each model's chosen confidence threshold. Fine-tuning reduces count MSE by roughly a factor of five over the pretrained YOLO baseline and more than a factor of ten over zero-shot SAM 3. The fine-tuned model is also nearly unbiased (-0.07 people/frame), while pretrained YOLO26n systematically under-counts (-1.63) and SAM 3 systematically over-counts ($+5.00$), hallucinating people in busy backgrounds (storefront mannequins, reflections, printed advertising).

Hyperparameter sweep summary (all 195 successful trials): Count MSE ranged from 2.79 (best) to 20.8 (worst); every `yolo26m` fine-tuning run in the final round was better than the pretrained baseline. The best-per-round MSE (v1 → v10) dropped from 7.06 to 2.79, indicating the search-space pruning strategy worked: later rounds reliably produced better configurations than earlier ones.

5.2 Compute Efficiency Results

Fine-tuning costs a one-time ∼10 minutes on a single GPU at our training settings. Inference cost is ∼30 ms/frame for the fine-tuned medium model vs. ∼14 ms/frame for the nano baseline and roughly half a second per frame for SAM 3 — in absolute terms, both YOLO variants are more than fast enough to run in real time on a 1 FPS livestream, while SAM 3 is order-of-magnitude slower and would become a bottleneck at higher frame rates. Figure 2 visualises the accuracy-vs-compute trade-off: the fine-tuned YOLO26m sits in the preferred corner of the plane — lower count MAE than either baseline *and* ∼15 × faster inference than SAM 3.

The full 26-dim sweep across 195 trials consumed roughly 5–7 minutes per trial on average, parallelised across two GPUs, for a total wall-clock cost under one day of compute.

Table 3: Compute efficiency comparison. Training time is per-trial; inference is per-frame at `imgsz=1280` on a single GPU.

Model	Train time (best trial)	Inference latency
Pretrained YOLO26n (baseline)	—	$\sim$14 ms/frame
SAM 3 zero-shot	—	$\sim$500 ms/frame
Fine-tuned YOLO26m (ours)	$\sim$10 min	$\sim$32 ms/frame

5.3 Trend Analysis Results

Applying the fine-tuned model to an unlabelled 1 FPS capture of Toomer's Corner recovers an intuitive crowd-count time series: counts are low during mid-morning class hours, rise steadily at lunchtime, and peak during the late afternoon as classes let out. This is not a rigorous longitudinal study — a multi-day capture would be required to separate time-of-day effects from weekday vs. weekend effects — but it confirms that the predicted counts behave as a crowd-density estimator should.

5.4 Qualitative Examples

In sparse scenes, both models detect the visible pedestrians correctly; the gap between baseline and fine-tuned is invisible to the eye. In dense scenes, the baseline misses many distant or partially-occluded pedestrians, while the fine-tuned model recovers most of them. Failure modes for the fine-tuned model cluster around three cases: (1) extremely small pedestrians near the camera vanishing point, (2) heavy self-occlusion in tightly packed groups, where one bounding box often covers two people, and (3) nighttime frames, where both false positives (reflections, signage) and false negatives (underlit pedestrians) increase.

6 Discussion

6.1 What Worked Well

The single strongest finding is that a relatively small curated dataset ($\sim$500 frames) is enough to turn a generic pedestrian detector into a reliable crowd counter for this scene. Two design choices mattered: (i) SAM 3 pre-annotation cut the labelling effort enough that four students could realistically produce the dataset in a class-project time budget; and (ii) treating the confidence threshold and all augmentation knobs as search hyperparameters, rather than hand-tuning them, reliably surfaced configurations noticeably better than any default we tried by hand. Making *count MSE* (not mAP) the sweep's selection metric also mattered — we observed several trials with higher mAP_{50} but worse counting, because mAP rewards tight localisation but not box-count fidelity.

6.2 Limitations

- **Single scene.** The model is fine-tuned on one fixed camera angle. We do not expect it to transfer to other streams without re-labelling.
- **Label noise.** Because annotation is based on SAM 3 pre-labels, systematically hard cases for SAM 3 (e.g. very distant or low-contrast pedestrians) are under-represented in the human-added corrections, and some residual labelling noise survives into the training set.
- **Small test set.** With only 101 validation images, count MSE differences below roughly 0.3 are within trial-to-trial noise, so we cannot reliably distinguish the top few sweep trials from one another.
- **Day-dominated data.** Most captured frames are daytime; the model is noticeably less reliable at night (see below).
- **Counting via boxes.** The detection-then-count pipeline inherits all standard detection failure modes (NMS merging two people into one box, double-counting a person split across occlusion), which a density-map approach might sidestep — we did not compare to such a method.

6.3 Sources of Error and Bias

Partial people at the frame border are sometimes labelled and sometimes not, which reviewers tried to handle consistently but could not fully eliminate; this is a measurable source of labeller disagreement. Dense crowds produce near-duplicate bounding boxes post-NMS that can collapse multiple nearby people into a single detection. Weather and night-time frames were deliberately kept in the set but are under-represented, meaning the model's night-time count variance is higher than daytime. Finally, because the SAM 3 pre-annotator biases what reviewers see first, there is a small but nonzero risk of confirmation bias in the labels — reviewers are more likely to approve an existing box than to notice a genuinely missed pedestrian in a visually busy region.

7 Conclusion

We built a person-counting system for the Toomer's Corner livestream end-to-end: frame extraction, semi-automatic annotation with SAM 3 + Label Studio, fine-tuning in the YOLO26 family, and a large hyperparameter search whose primary metric is count MSE rather than mAP. Fine-tuning cut per-image count MAE to 1.27 (fine-tuned YOLO26m) from 3.10 (pretrained YOLO26n) and 5.00 (SAM 3 zero-shot), and count MSE to 2.79 from 15.01 and 33.09 respectively, on the same held-out validation split. The fine-tuned model is essentially unbiased; the pretrained YOLO baseline systematically under-counts and SAM 3 systematically over-counts. Compute costs are negligible (one-time $\sim$10 min fine-tuning, $\sim$30 ms/frame inference), so this approach is viable as an always-on counter. Natural next steps include multi-scene transfer, density-map methods for very dense frames, and a larger longitudinal capture to quantify time-of-day and weekday-vs-weekend effects rigorously.

8 Team Contributions

- **Jase Schwanke:** Frame extraction pipeline (`extract_frames.py`), livestream sampling strategy, qualitative failure-case curation, writing contributions to the dataset and discussion sections.
- **Logan Bolton:** SAM 3 annotation pipeline (`sam3/annotate.py`, `sam3/review.py`) and Label Studio integration, `sweep.py` implementation and multi-GPU orchestration, final sweep analysis.
- **Sathvik Prahadeeswaran:** Baseline YOLO evaluation, SAM 3 zero-shot counting evaluation, hyperparameter search-space pruning between sweep rounds, compute-efficiency measurements.
- **Trey Wendell:** Dataset splitting (`prepare_dataset.py`), annotation review shifts, `compare_models.ipynb` for final cross-model plots, writing contributions to the introduction, methods, and conclusion.

All authors participated in labelling shifts in Label Studio and contributed to reviewing the final report.

Code and Reproducibility

The project repository contains everything needed to reproduce the results. Entry points:

- `extract_frames.py` — sample frames from the livestream.
- `sam3/annotate.py` — generate raw SAM 3 labels.
- `sam3/review.py` — launch Label Studio for human correction.
- `prepare_dataset.py` — build train/val split and `dataset.yaml`.
- `train.py` — single fine-tuning run.
- `sweep.py` — multi-GPU random hyperparameter search. Results land in `sweeps/<name>/results.csv`.
- `compare_models.ipynb` — side-by-side count metrics, best-confidence-threshold selection, and plots used in this report.

Environment is managed with `uv` (`pyproject.toml` + `uv.lock`); a single `uv sync` pins all dependencies. All randomness uses fixed seeds (`prepare_dataset.py` seed $= 42$; sweep workers seed from a reproducible base). Best model weights for each retained trial are stored under `sweeps/<name>/trials/<trial_id>/best.pt`. See the repository `README.md` for a quickstart.

References

- Ultralytics, *YOLO26*. `https://docs.ultralytics.com/`
- A. Kirillov et al., *Segment Anything*, ICCV 2023; SAM 3 open-vocabulary extension.
- Label Studio (Heartex), `https://labelstud.io/`
- T.-Y. Lin et al., *Microsoft COCO: Common Objects in Context*, ECCV 2014.